Arithmos Calculator — DevOps Pipeline Mini-Project Report

Kaushalraj Puwar

October 2026

1 Overview

Arithmos is a small but production-style Calculator REST API in Java 21 (Spring Boot, Maven-built executable JAR), wrapped in a fully automated DevOps pipeline using only the pre-midterm toolchain: Git/GitHub for version control, Jenkins for CI/CD, Docker for containerization, and Docker Compose for deployment. A push to `main` triggers a pipeline that builds, tests, packages, containerizes, pushes, and deploys with no manual steps.

Links: GitHub repository `https://github.com/KaushalrajPuwar/arithmos-pipeline` Docker Hub image `https://hub.docker.com/r/kaushalrajpuwar/arithmos-calculator`

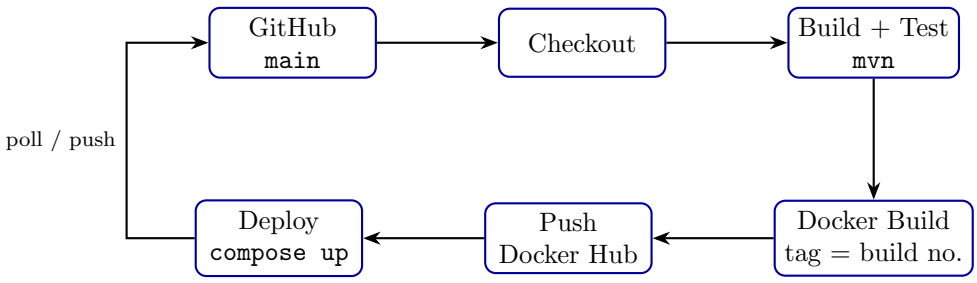
2 Application

Four endpoints under `/api/v1` (`add`, `subtract`, `multiply`, `divide`) accept numeric `a`, `b` query parameters and return `{"operation","a","b","result"}` as JSON. Division by zero and invalid or missing parameters return HTTP 400 with a structured error body (`timestamp`, `status`, `error`, `message`, `path`) produced by a global exception handler. Spring Actuator exposes `/actuator/health` (UP/DOWN), which the container runtime uses as the Compose health check.

3 Testing

JUnit 5 service tests cover every operation including the divide-by-zero case (now 8 tests after a decimal-addition case was added as the live-demo change), plus 7 Spring MockMvc web-slice tests that hit real endpoints and assert the JSON contract and both 400 error paths. All 15 tests run via `mvn test` and emit JUnit XML under `target/surefire-reports/`, which Jenkins publishes with the `junit` step and fails the build on any failure.

4 Pipeline architecture



The declarative `Jenkinsfile` defines six stages. **Checkout** pulls the latest `main`. **Build** runs `mvn clean package -DskipTests` pinned to Homebrew Java 21. **Test** runs `mvn test` and archives JUnit XML. **Docker Build** builds the multi-stage image tagged with both the Jenkins build number and `latest`. **Push** logs in with the Jenkins-managed `dockerhub-creds` credential (never hard-coded) and pushes both tags. **Deploy** runs `docker compose down --remove-orphans` then `docker`

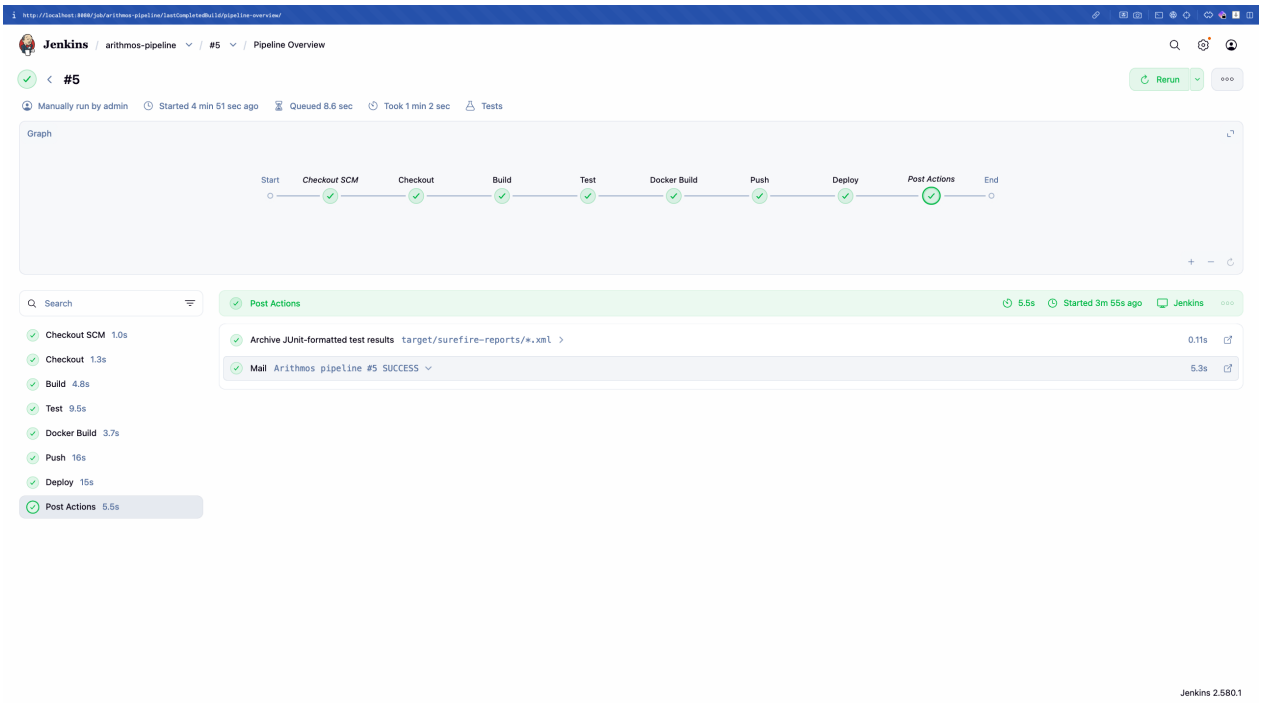
`compose up -d` (idempotent re-runs), waits for `/actuator/health` to report UP, then verifies an end-to-end call through the Nginx proxy. Triggers are `githubPush()` with `pollSCM('H/2 * * * *')` as fallback, and every build sends a success/failure email via Gmail SMTP. Build #6 proved the loop: a test commit pushed from a laptop was picked up automatically and deployed.

5 Docker and Compose

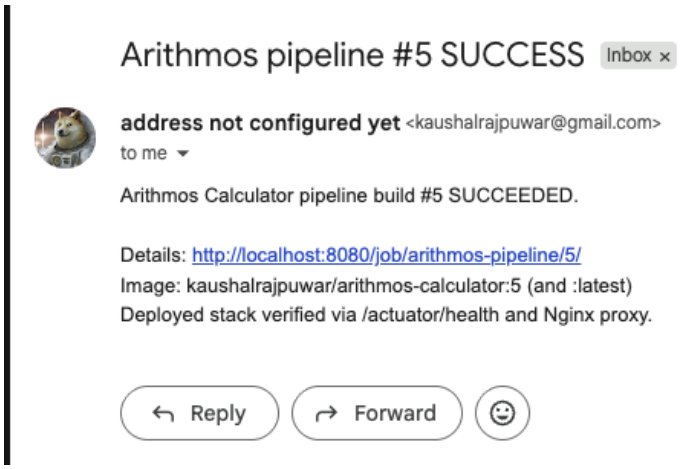
The multi-stage `Dockerfile` compiles in a Maven/JDK 21 stage (dependencies copied first for layer caching) and copies only the JAR into a slim JRE Alpine runtime that runs as non-root user `app` with an explicit `EXPOSE 8080`. A single `docker run -p 8080:8080` serves the API. `docker-compose.yml` orchestrates two containers on one network: `calculator` (host 8081, restart policy, Actuator health check, `APP_PORT` env support) and an Nginx reverse proxy (host 8082) that only starts once the calculator is `healthy`. One `docker compose up -d` launches the full stack.

6 Evidence

Build #5 (and later auto-triggered build #6) ran fully green: all six stages plus JUnit archiving and the success email. The deployed stack answers through both the proxy (`:8082`) and the app directly (`:8081`), with divide-by-zero correctly returning HTTP 400.



Green pipeline run (build #5): all stages and post actions green.



Build status email notification.

```
❯ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES                STATUS                    PORTS
arithmos-proxy       Up 4 minutes              0.0.0.0:8082→80/tcp
arithmos-calculator  Up 5 minutes (healthy)    0.0.0.0:8081→8080/tcp
```

`docker ps`: both containers up, calculator healthy.

```
kaushalrajpuwar ~ 20:31
❯ curl "http://localhost:8082/api/v1/add?a=2&b=3"
{"operation":"add","a":2.0,"b":3.0,"result":5.0}%

kaushalrajpuwar ~ 20:32
❯ curl "http://localhost:8082/api/v1/divide?a=5&b=0"
{"timestamp":"2026-10-30T15:02:20.454034845Z","status":400,"error":"Bad Request","messag
e":"Division by zero is not allowed","path":"/api/v1/divide"}%

kaushalrajpuwar ~ 20:32
❯ curl http://localhost:8081/actuator/health
{"status":"UP","groups":["liveness","readiness"]}%
```

Live API: add via proxy, divide-by-zero 400, actuator health.

7 Challenges faced

Java version skew: Jenkins' default `PATH` resolved Java 27 while the project targets 21, and Mockito's inline mock maker cannot run on 27 — all MockMvc tests errored. Fixed by pinning `JAVA_HOME`/`PATH` to the Homebrew Java 21 install in the `Jenkinsfile` environment block. **Deploy conflicts:** leftover containers from manual testing collided with Jenkins' `compose up` (same container names, different Compose project directories). Fixed by making Deploy idempotent with `down --remove-orphans` first. **Email:** no SMTP server listened on `localhost:25`; configured the Jenkins mailer for Gmail SMTP (port 587, app password held in the Jenkins credentials store, never committed) and verified delivery. **Webhook:** GitHub rejects `localhost` webhook URLs, and no tunnel tool was available, so GitHub-to-Jenkins push delivery relies on the spec-allowed Poll SCM fallback while `githubPush()` stays configured for a public Jenkins URL. No credentials or secrets were ever committed; the repository history (feature branch + pull request + progressive fixes) is intentionally granular.